OL Report

Student
GT ID: x

Abstract— **Understanding training configurations and optimization choices is crucial when working with machine learning models. This report investigates the impact of randomized optimizations, Adam ablation tests, and various regularization techniques on neural network performance. I analyze and evaluate their specific contributions to model behavior and generalization.**

I. DATASETS

A. Adult Income

The adult income data is extracted from 1994 census database [1]. The prediction task is a binary classification problem to predict whether a person's income is above or below 50k. The raw dataset has 45222 rows and 15 columns without any missing data. Number of rows decreased to 45175 after de-duplication. The dataset contains 6 numeric columns and 8 categorical columns. The "class" column is the target label with 2 unique values: income >50k (25%) and <=50k (75%). No data leakage was found. All data cleaning and pre-processing were kept the same as SL report. "education-num", "capital-gain" and "capital-loss" columns were dropped. One hot encoding was applied to all categorical features expect for "native-country" where a frequency encoding was applied. A standard scaler was applied to all numeric columns including frequency encoded column. One hot encoded columns were left as 0s and 1s. Consistent with the SL report, the full dataset was first split into 80% training set (n=36140) and 20% test set (n=9035), then additional 20% was held out from training set as validation set (n=7228). All splits were stratified by target label.

B. Wine Quality

The wine quality dataset measures physicochemical properties of white and red wine [2]. The prediction task is a multi class problem to predict the quality class of a wine. The raw dataset has 6497 rows and 14 columns without any missing data. Number of rows decreased to 5320 after de-duplication. The dataset contains 11 columns of float64 data type and 3 columns of int64 data type. "quality" column is the target label with 8 unique categories ranging from 1 to 8. Target 4 (35%) is the most frequent class. All data cleaning and pre-processing were kept the same as SL report. "class" column was dropped. "type" column was converted to categorical datatype and one hot encoding was applied. Standard scaler was applied to all numerical features. Consistent with the SL report, the full dataset was first split into 80% training set (n=4256) and 20% test set (n=1064), then additional 20% was held out from training set as validation set (n=852). All splits were stratified by target label.

II. HYPOTHESIS

A. Randomized optimization

Three randomized optimization algorithms: Random Hill Climb (RHC), Simulated Annealing (SA) and Genetic Algorithm (GA) will be applied on both adult income and wine quality dataset. My hypothesis is that SA will have very little to no improvement in loss during the initial exploratory high temperature phase. But loss decrease will be more obvious during later exploitation phase with lower temperature. Therefore, overall convergence is expected to be slow for SA. RHC will likely show steady decrease in loss due to it's greedy search methodology, therefore convergence is expected to be faster than SA. Best validation loss reached for SA and RHC depends on data complexity. For noisy and ill conditioned landscape, SA may reach lower validation than RHC due to it's early exploratory phase. Otherwise, I hypothesize both algorithms to have similar best validation loss reached. Genetic Algorithm update model states based on population fitness, selecting only high fitness individuals for crossover. Therefore I hypothesize that it will exhibit stable and rapid convergence, resulting in the best validation loss among three RO algorithms.

B. Adam ablation

For gradient based optimizers, SGD without momentum would show slow convergence because weight updates are based solely on learning rate and current gradients. SGD with momentum (heavy ball) introduces velocity term, which enables faster convergence. The momentum term can also improve stability in high variance regions. SGD with Nesterov momentum further incorporate look ahead term to mitigate overshoot and reduce oscillation. My hypothesis is that SGD with Nesterov momentum will have the most stable and fast convergence among three gradient based optimizers.

AdaGrad and RMSprop both implement adaptive preconditioning that scale updates for each parameter. The difference is that AdaGrad accumulates all gradient history whereas RMSprop let old gradients fade and adapts to recent gradient scale. My hypothesis is that AdaGrad may take longer to converge than RMSProp but will reach a lower loss because it is well suited for sparse features according to professor LaGrow [3]. Adam combines RMSProp and momentum, and is expected to have fastest convergence. Adam without bias correction may have slow start due to initial bias towards 0, thus may exhibit slower convergence and ending with higher loss. Adam with regularization will also have higher loss but is expected to have best generalization.

C. Regularization

Regularization is a commonly used technique to reduce model overfitting. I hypothesize that regularization methods will provide limited improvement in model generalization on adult income dataset. One reason is that the dataset have 3:1 class imbalance, which poses a natural challenge for model generalization. Additionally, results from training in the SL report indicates no severe overfitting pattern was observed when using vanilla SGD without momentum as optimizer. Since regularization will be applied in conjunction with Adam optimizer in this report, which is generally better at navigating ravines than SGD without momentum, further reduction in overfitting is unlikely.

III. Baseline Model

The better performing SL backbone NN models for both datasets are the wide NN with 3 hidden layers, which were implemented with SKlearn in SL report. They are replicated within PyTorch using the exact same architecture and hyperparameters to establish the baseline for this report. The baseline NN models for both datasets were trained using SGD optimizer without momentum, without weight decay, with tuned learning rate for specific number of epochs. For the adult income dataset, the model was trained over learning rate of 0.008 for 20 epochs, achieving F1 score of 0.64 and accuracy of 0.84 on test set. For the wine quality dataset, the model was trained over learning rate of 0.03 for 50 epochs, achieving macro F1 score of 0.34 and accuracy of 0.57 on test set.

IV. Randomized Optimization

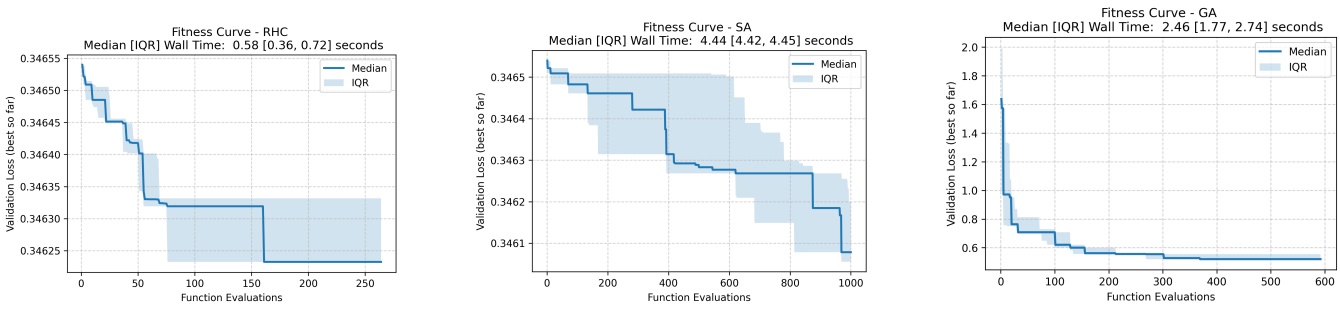
Three randomized optimization algorithms (RHC, SA, GA) were implemented using the mlrose-hiive library. Model states were updated based on training data, and validation loss was computed using updated model states as the primary evaluation objective. To assess the effect of RO on improving model tail behavior, the parameters of the first hidden layer were frozen and fixed to the SL backbone trained values. The parameters for last two hidden layers were optimized using RO algorithms. A computing budget of 1000 function evaluations was imposed on all three algorithms to ensure fair comparison. Initial parameter exploration was conducted through a pilot run with single seed. Final runs were performed with selected parameters across five seeds to evaluate performance stability.

A. Adult Income

The SL backbone NN ($61 \rightarrow 50$, 40, $30 \rightarrow 1$) has 6401 total training parameters. After freezing first hidden layer, the number of trainable parameters decreased to 3301. For RHC of 5 runs (1 initial run + 4 restarts), the fitness curve (Figure 1a) shows a steady decrease in validation loss with low variation among seeds early on. After around 70 function evaluations, the curve plateaus and the IQR widens, indicating increased variability. A further validation loss drop around 160 function evaluations suggests that some runs found better solutions while most remain stuck at local minima. All runs converged by 260 function evaluations with a wall time of 0.72 seconds. For SA (Figure 1b), validation loss decreases slowly initially, followed by drops around 400 and 900 function evaluations. The IQR band shows decrease in validation loss after 600 function evaluations with big variation among seeds. SA did not converge within specified budget, with a median wall time of 4.4 seconds for 1000 function evaluations. Overall, both RHC and SA yield small loss reduction and limited performance improvement. On test set, RHC achieved median F1 of 0.65 [IQR: 0.6441, 0.6468] and median accuracy of 0.84 [IQR: 0.8380, 0.8386], SA achieved median F1 of 0.65 [IQR: 0.6475, 0.6480] and median accuracy of 0.84 [IQR: 0.8382, 0.8384]. Both are almost identical to baseline model performance. For GA (Figure 1c), validation loss decrease rapidly and converges within the budget, with a narrow IQR indicating stable performance across seeds. Although GA achieves the largest overall decrease in validation loss, it's best validation loss remains higher than that of RHC and SA. On test set, GA underperforms the baseline with a median F1 of 0.52 [IQR: 0.4835, 0.5535] and median accuracy of 0.77 [IQR: 0.7354, 0.7926].

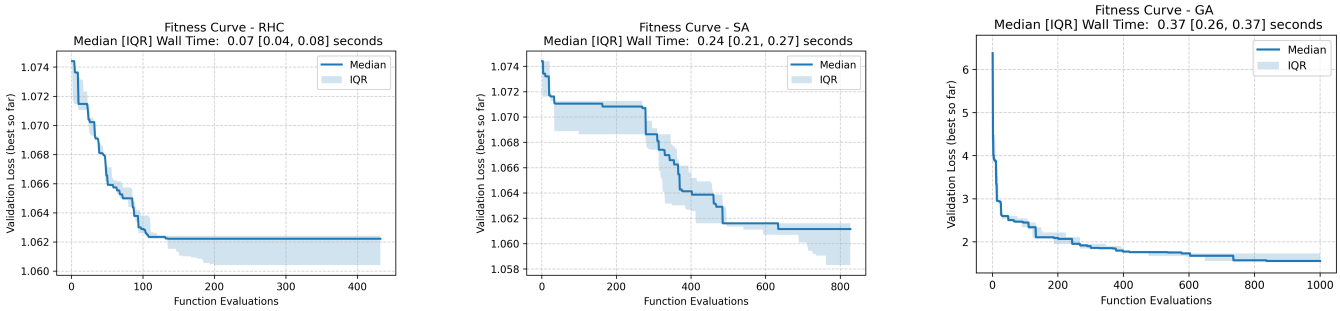
B. Wine Quality

The SL backbone NN ($13 \rightarrow 14$, 16, $10 \rightarrow 8$) has 694 total training parameters. After freezing first hidden layer, the number of trainable parameters decreased to 498. For RHC, a smaller step size (0.05) was selected as bigger step size tend to cause stagnation in wine quality dataset. The fitness curve (Figure 2a) shows rapid and steady loss decrease, converging after around 400 function evaluations with 0.07 seconds. The IQR band is narrow before 120 function evaluations but widens afterwards indicating some runs continue finding better solutions. For SA (Figure 2b), median validation loss decreased during initial high temperature phase, plateaus, then drops again around 280 function evaluations. The median loss stabilizes after around 620 function evaluations though some runs continue to improve. The loss converged after around 800 function evaluations with 0.2 seconds, indicating slower convergence than RHC. Both algorithms reach similar best validation loss, but overall performance improvement is limited. On test set, RHC achieved a median macro F1 of 0.33 [IQR: 0.3323, 0.3358] and median accuracy of 0.57 [IQR: 0.5686, 0.5695], SA reached a median macro F1 of 0.33 [IQR: 0.3234, 0.3286] and median accuracy of 0.57 [IQR: 0.5724, 0.5771], both nearly identical to baseline performance. For GA, a moderate population size and small mutation probability produced stable results. The fitness curve (Figure 2c) shows rapid loss decrease and near converge at 1000 function evaluations with 0.37 seconds. Even though GA achieves larger overall loss reduction, it's best validation loss remains higher than RHC and SA due to random initialization. On test set, GA underperforms baseline with a median macro F1 of 0.16 [IQR: 0.1546, 0.1568] and median accuracy 0.34 [IQR: 0.3244, 0.3582].



(a) (b) (c)

Fig. 1: Adult Income Dataset Randomized Optimization. Function evaluation budget=1000 and max attempt=10 for all 3 algorithms. For RHC, 5 total runs, step size=0.1. For SA, step size=0.1, initial temperate=50, decay=0.9. For GA, population size=20, mutation probability=0.2



(a) (b) (c)

Fig. 2: Wine Quality Dataset Randomized Optimization. Function evaluation budget=1000 and max attempt=10 for all 3 algorithms. For RHC: step size=0.05, total 5 runs. For SA, step size=0.1, initial temperate=100, decay=0.9. For GA, population size=20, mutation probability=0.05

Dataset	RO Algorithm	Best Val Loss	Test F1	Test Accuracy	Function Evaluations
Adult income	RHC	0.346	0.645	0.839	126
Adult income	SA	0.346	0.648	0.838	1000
Adult income	GA	0.520	0.520	0.766	510
Wine Quality	RHC	1.062	0.334	0.570	130
Wine Quality	SA	1.061	0.329	0.573	635
Wine Quality	GA	1.554	0.157	0.336	1000

TABLE I: RO summary table for both datasets. Median values from 5 seeded runs are shown for best validation loss, test F1, test accuracy, function evaluations

C. Summary

In both datasets, the findings are consistent with my hypothesis that RHC converges faster than SA. And both algorithms reach similar best validation loss and achieve similar performance on test set. GA achieves most rapid and largest overall loss reduction but fails to match baseline performance, which contradicts my initial hypothesis. A likely explanation is the difference in initialization, RHC and SA explore neighbors of the trained baseline weights, whereas GA initializes population with randomly generated weights. Although weights were constrained to range [-1,1] to start closer to weights obtained from trained baseline model, GA was not able to evolve beyond the performance that has already been achieved.

V. Adam Ablations

The Adam optimizer, introduced in 2014, has become a popular optimization method due to it's strong performance compared with traditional stochastic optimization methods [4]. In this section, I examine seven optimizer variations (SGD no momentum, SGD with momentum, SGD with Nesterov momentum, base Adam, Adam without bias correction, RMSProp-like Adam and AdamW) applied to the adult income data under same computing budget to compare their behavior.

The train (n=28912) and validation (n=7228) split, as well as the batch size (n=32) were kept the same for all seven optimizer variations. The primary computing budget was the number of gradient evaluations, computed per batch. Parameter search was performed in a pilot run using a smaller budget of 1000 gradient evaluations, with three seeded runs performed independently for each optimizer variation. After selecting the final parameters, five seeded runs with 5000 gradient evaluations each were conducted to compare the resulting loss trajectories for seven optimizer variations.

To evaluate optimizer performance, a loss target of ℓ=0.363 was defined as 75th percentile of the mean validation loss obtained from three seeded runs using the base Adam optimizer with default parameters over 1000 gradient evaluations. Time and steps to reach ℓ were recorded for each optimizer variation, as well as test F1 score and accuracy.

A. SGD Family

SGD without momentum is the same optimizer used in the SL report; therefore the same tuned parameters were retained: learning rate=0.008 and weight decay=0. As shown in Figure 4a, validation loss exhibits a rapid initial decrease during the early training phase, indicating efficient progress toward lower loss region in the error landscape. This is followed by a period of slower improvement approximately between 1000 and 2000 gradient evaluations, where loss curve flattens and more variation occurred. This suggests reduced optimization efficiency and possible traversal in less curvy region. After around 2000 gradient evaluations, validation loss continues to decrease until computing budget was reached. The loss did not converge by the end of the budget as indicated by downward trending curve at the final portion of the trajectory.

For SGD with momentum, a parameter sweep was conducted over learning rates $\in \{0.001, 0.005, 0.008\}$, momentum $\in \{0.8, 0.9, 0.98\}$. Smaller learning rate and momentum combination was found to produce smoother loss trajectory, whereas larger values introduced noticeable oscillation. Based on these observations, the final configuration was set to a learning rate of 0.005 with momentum 0.8. The same parameters were used for Nesterov momentum to isolate the effect of look-ahead term.

Compared to SGD without momentum (Figure 4a), both momentum based variants traversed regions of slow loss decrease more quickly and achieved faster convergence. In particular, they exhibit steady loss reduction between 500 to 2500 gradient evaluations where SGD without momentum experienced prolonged plateau. Variation across different seeds is more pronounced between 1000 to 2000 gradient evaluations, as indicated by wider IQR band.

Contrary to my initial hypothesis, SGD with momentum and with Nesterov momentum produce nearly identical loss trajectories. This suggests that the error landscape may be well conditioned, limiting the benefit of Nesterov look ahead correction.

B. Adam Family

Parameter sweep was conducted for Adam family optimizers over learning rates $\in \{0.0001, 0.001, 0.01\}$, $\beta1 \in \{0.8, 0.85, 0.9\}$, and $\beta2 \in \{0.99, 0.995, 0.999\}$. The mean validation loss after 1000 gradient evaluations, averaged over three seeded runs was used to visualize sensitivity heatmaps. As shown in Figure 3, a learning rate of 0.001 consistently produced lowest validation loss across all Adam family variations. When the learning rate was very small (0.0001), the resulting validation losses were higher overall, and differences among various beta values were more pronounced. In contrast, at larger learning rates, the variation in validation loss was dominated by the learning rate itself, while the impact of different beta values was minor. This observation indicates that the dataset is more sensitive to learning rate than to the exponential moving average parameters that control the effective "memory" of past gradients.

Based on parameter sweeps, the final parameters were chosen as follows: Adam with $\beta1$ = 0.85, $\beta2$ = 0.999; Adam without bias correction with $\beta1$ = 0.8, $\beta2$ = 0.995; RMSProp-like Adam with $\beta1$ = 0, $\beta2$ = 0.999; and AdamW with $\beta1$ = 0.85, $\beta2$ = 0.999. Using these tuned parameters, the four Adam family variants exhibit very similar loss trajectories in Figure 4. All methods achieved rapid validation loss reduction in first few hundred gradient evaluations, after which the loss stabilizes and remains nearly constant for the remainder of the training budget.

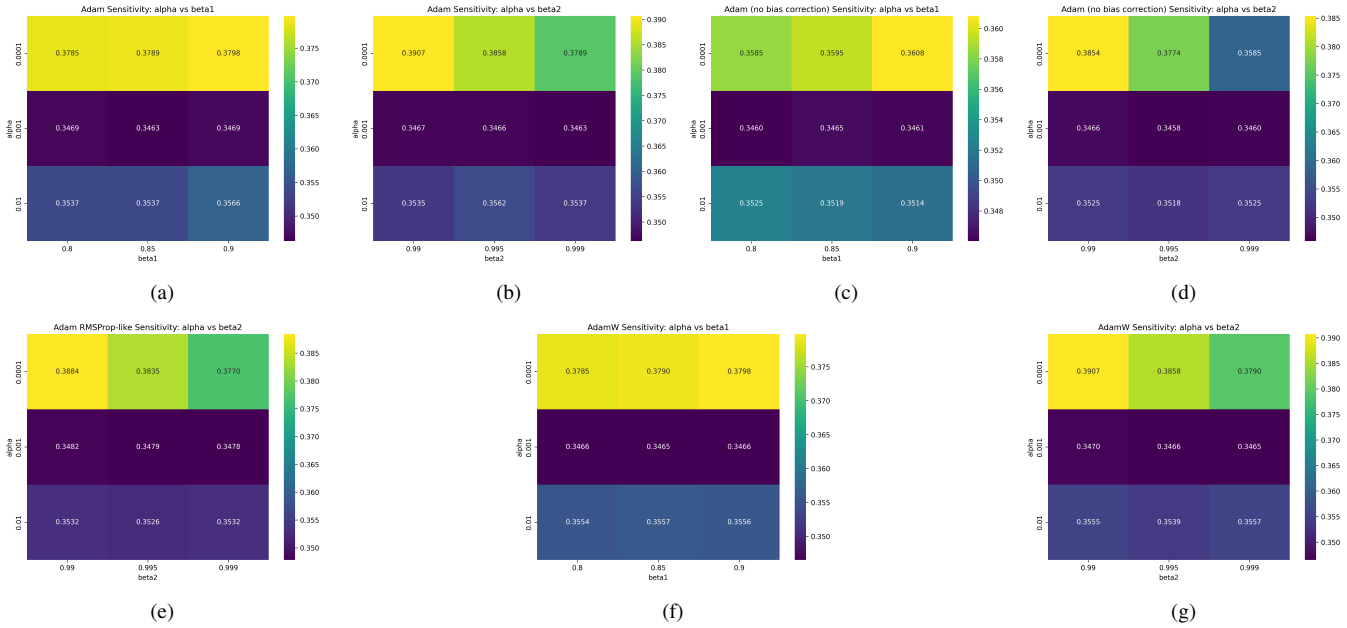
Among the variants, Adam without bias and RMSProp-like Adam show a slightly faster loss decrease at the beginning of training compared to Adam and AdamW, which contradicts my hypothesis that Adam would converge fastest. This may suggest that the dataset is insensitive to the momentum component controlled by $\beta1$. For Adam without bias, the steeper initial decrease can be explained by the inflated step size at the beginning of training. When moving averages are initialized to 0, the absence of bias correction result in an scaling factor of

$$\frac{1-\beta_1}{1-\beta_2} = \frac{1-0.8}{1-0.995} = 40$$

, which temporarily increase the update magnitude. As training proceeds and moving averages accumulate, the step size become more stable, leading to similar behavior to other Adam variants.

C. Optimizer Comparison

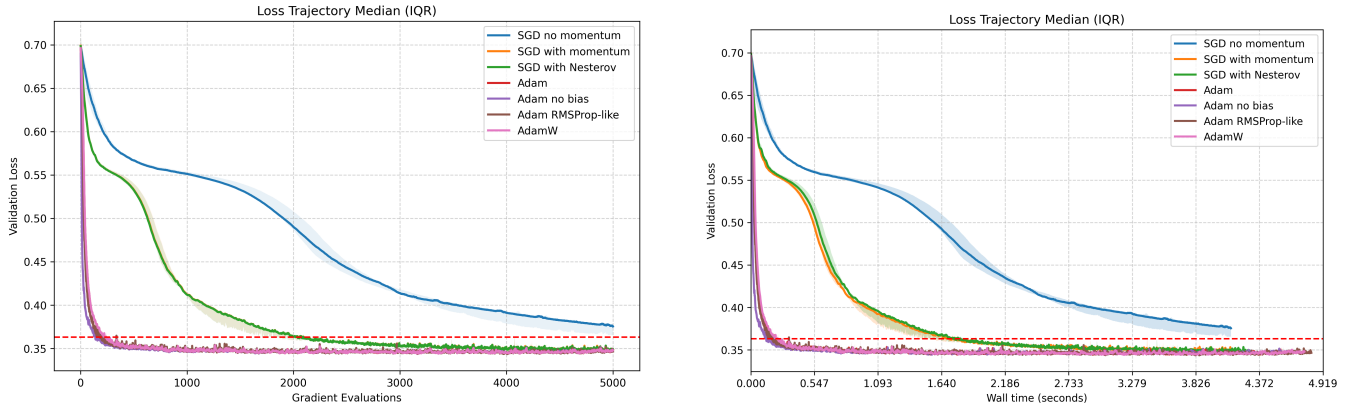
SGD without momentum converges more slowly and maintains higher validation loss during training. The loss decrease gradually and did not reach target loss ℓ within budget. This behavior highlights the limited optimization speed of vanilla SGD. SGD with momentum and with Nesterov momentum both converge faster than SGD without momentum and eventually reach target loss. However, their convergence still remain slower than that of adaptive optimizers such as Adam and its variants. As shown in Table II and Figure 4b, Adam family optimizers require substantially fewer steps (137 to 231) and time (less than 0.5 second)



(a) (b) (c) (d)

(e) (f) (g)

Fig. 3: Sensitivity heatmaps for Adam family optimizers on adult income dataset



(a) (b)

Fig. 4: Loss trajectories for 7 optimizer variations

to reach ℓ than SGD with momentum and with Nesterov momentum, which require over 2000 steps and more than 1.5 seconds. This behavior is consistent with rapid early loss decrease observed for Adam family optimizers.

Performance on test set follow a similar pattern. SGD no momentum achieves the lowest test F1 score of 0.601, which is expected due to slow training progress. Adding momentum improved test F1 score to 0.637, which is comparable to SL baseline test F1 score of 0.64. Adam family optimizers achieve slightly higher test F1 score though the improvement is modest. Among all optimizer variants, RMSprop-like Adam achieves the highest test F1 score of 0.657, suggesting that removing first momentum does not harm, instead slightly improve generalization for this dataset.

Through examining the relationship between training and validation dynamic in Figure 5 reveals that final validation loss of most variants stabilize around 0.35. However instantaneous loss during training exhibits noticeable fluctuations, also shown by the jagged patterns in Figure 4. These variations arise from model updates based on small batch size of 32 samples, which introduces stochastic noise in gradient estimates, leading to local fluctuations in the loss values.

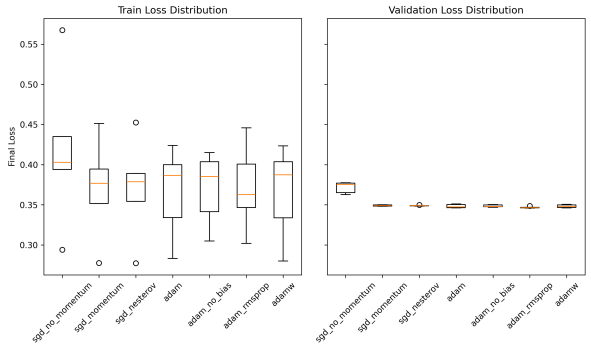
Overall, the results indicate that adaptive optimizers provide faster speed to target while yielding similar or slightly better generalization results compared to SGD based optimizers. The relatively small difference also suggests that the dataset presents a relatively well conditioned error surface where many optimizers ultimately achieve similar performance once sufficiently trained.

VI. Regularization

Regularization is critical for mitigate overfitting and enhancing model generalization. In this section, I evaluate

Method	Best Val Loss	Test F1	Test Accuracy	Steps to ℓ	Time to ℓ	Grad Evals	Updates
SGD no momentum	0.375	0.601	0.827	did not reach	did not reach	5000	5000
SGD with momentum	0.347	0.637	0.837	2021	0.36 seconds	5000	5000
SGD Nesterov momentum	0.347	0.637	0.837	2020	0.36 seconds	5000	5000
Adam	0.343	0.648	0.839	224	0.36 seconds	5000	5000
Adam no bias correction	0.344	0.647	0.840	137	0.36 seconds	5000	5000
Adam RMSprop-like	0.343	0.657	0.838	184	0.36 seconds	5000	5000
AdamW	0.344	0.647	0.840	231	0.36 seconds	5000	5000

TABLE II: Adam ablation summary table. Median values from 5 seeded runs are shown for best validation loss, test F1, test accuracy, steps and time to reach ℓ



(a)

Fig. 5: Generalization gap. box plot of last train and validation loss from each seeded run.

five different regularization techniques with the objective to understand the improvement in model generalization yielded by each technique. Parameter search was conducted using training and validation set. Due to the imbalanced target distribution in the adult income dataset, best validation loss and test F1 score are selected as primary metrics for evaluating model generalization. Test accuracy is also reported to assess generalization accuracy tradeoff balance. Results are compared against baseline model trained with base Adam (learning rate = 0.001, $\beta 1$ = 0.85, $\beta 2$ = 0.999) in previous section.

A. L2 weight decay (coupled)

L2 weight decay reduce model overfitting by adding a penalty proportional to squared weights in update steps. When using Adam optimizer, L2 weight decay can be implemented as decoupled (AdamW) or coupled; this section focuses on the latter. The hyperparameter λ controls regularization strength with larger values penalize weights more. After comparing $\lambda \in \{1e{-}6, 1e{-}5, 1e{-}4, 1e{-}3, 1e{-}2, 1e{-}1\}$, I found that high λ (>= 0.01) led to elevated validation loss, suggesting that weight updates were too conservative. While smaller λ (< 0.001) achieved comparable best validation loss. Based on this observation, λ was configured to be 0.001 and achieved test F1 score of 0.658 and accuracy of 0.837. Through scaling down weight updates, the model achieves better generalization by having higher test F1 score than base Adam optimizer.

B. Early Stopping

Early stopping is a regularization technique to prevent overfitting by halting training process when validation loss stops showing significant improvement. In this section, the significant improvement was defined as a decrease of at least 0.001 relative to best so far validation loss. Patience is the key hyperparameter for early stopping, and values of 100, 500, 1000, 2000 gradient evaluations were explored. Lower patience thresholds (100 and 500) led to premature stop at elevated validation loss, whereas higher patience thresholds (1000 and 2000) achieved identical best validation loss of 0.343. The final patience was configured to be 1000 gradient evaluations since training for additional 1000 steps did not provide further improvement. The resulting model achieved test F1 score of 0.653, slightly outperforming the base Adam by preventing the model from fitting to stochastic noise in the training data.

C. Dropout

Applying dropout layers is another common method of regularization. The method works by deactivating a fraction of neurons within a layer during each training iteration. In the architecture of three hidden layers, dropout layers were implemented following the first and second hidden layer, with dropout rates denoted as $p1$ and $p2$ respectively. Dropout for the third hidden layer was avoided as it may alter output logits and cause excessive perturbation. A grid search was conducted on different combinations of $p1$ and $p2 \in \{0, 0.3, 0.5, 0.7\}$. This range allowed for experimenting from no dropout (p=0) to high drop out rate (p=0.7). The results show that excessive dropout either too low or too high did not improve generalization. The best combination found was $p1$ = 0.3 and $p2$ = 0.5, which achieved a better test F1 score of 0.657 compared to that of base Adam while having comparable test accuracy. This improvement suggests that the dropout technique encouraged each neuron to learn robust independent features rather than replying on co-adaption [5].

D. Label smoothing

Label smoothing is an effective regularization method for classification problems and has been implemented in do-

mains such as image classification and language translation [6]. The method works by "softening" the traditional one-hot encoded target labels using a smoothing factor α. For a binary classification problem such as the adult income dataset, negative (0) labels would be converted to $0.5\times\alpha$ and positive (1) labels would be transformed to (1 - α) + $0.5\times\alpha$. A search grid for $\alpha \in \{0.05, 0.1, 0.15, 0.2\}$ was conducted on training target labels. The best α was found to be 0.15, resulting in smoothed targets of 0.075 for negative labels and 0.925 for positive labels. This configuration achieved better test F1 score of 0.660 compared to that of base Adam while having comparable test accuracy. This suggests that label smoothing mitigates overconfidence by preventing the model from making extreme predictions, which ultimately improves generalization.

E. Input noise injection

Input noise injection is a regularization method that introduces small perturbations to input data during training to enhance model robustness and reduce overfitting. One common approach is to add noise vector, sampled from zero mean Gaussian kernel, to the input data independently during each training iteration [7]. This approach is suitable for the adult income dataset as the numerical features were scaled to have mean of 0 and standard deviation of 1. For generating noise vectors, I evaluated Gaussian distribution with different standard deviation $\sigma \in \{0.01, 0.05, 0.1, 0.15, 0.2\}$. The results indicated that too small or too big σ either did not improve generalization or caused excessive perturbation. σ = 0.15 was found to be the sweet spot. With configured noise injection, the model achieved nearly identical test F1 score and slightly lower test accuracy than that of base Adam. This iterative injection of input noise challenges the model to learn general features rather than fitting to rigid details in the training data.

F. Regularization combination

To identify the best combination of the five regularization techniques, a two phase search strategy was implemented in lieu of an exhaustive grid search.

Phase 1: Start with all five methods each with tuned parameters $\rightarrow$ reduce one method at a time to observe their impact on generalization. As shown in Table IV(a), removing most methods resulted in decrease in test F1 score and increase in test accuracy, which indicates a widening generalization gap for imbalanced classification problem. In particular, removing label smoothing caused the most significant decrease in test F1 score. In addition, removing early stopping caused decrease in both test F1 and accuracy which suggests overall decrease in model performance. On the other hand, removing input noise resulted in an increase in both test F1 score and accuracy suggesting input noise injection inhibited model performance. Consequently, label smoothing and early stopping were identified as the key regularization methods while input noise injection was excluded from further consideration.

Phase 2: This phase focus on iteratively adding the remaining methods back to the key methods. Small parameter adjustments were performed around tuned value to observe change in generalization. As shown in Table IV (b), with label smoothing and early stopping, the test F1 score of 0.655 is already higher than base Adam test F1 score of 0.649. Extending the early stopping patience from 1000 to 2000 gradient evaluations further improved the test F1 score to 0.669. While adding more regularization methods consistently improved test F1 scores, they did not surpass the test F1 score of the 2 key method combination. However, by reducing coupled L2 weight decay from 0.001 to 0.0001, a 4 method combination was found to have similar test F1 score of 0.670 compared to the 2 key method combination.

G. Summary

Among the five regularization techniques, label smoothing and coupled L2 weight decay emerged as top two individual performer, yielding the highest improvement on test F1 score. These two methods improved generalization by mitigating overconfidence and constraining weight magnitudes without impairing test accuracy. Moreover, the combination of label smoothing and early stopping at 2000 gradient evaluations provided the most efficient regularization combination that boosted test F1 score to 0.669. Contrary to the initial hypothesis that regularization would provide limited improvement to model generalization, this analysis demonstrates that strategic implementations of regularization techniques are effective for addressing generalization gaps by improving F1 scores while maintaining accuracy in imbalanced classification tasks.

VII. AI USE STATEMENT

I used Gemini and VS Code to assist with python code generation, debugging, grammer correction, overleaf figure and table formatting. I reviewed, verified and understand all assisted content.

REFERENCES

[1] B. Becker and R. Kohavi, "Adult." UCI Machine Learning Repository, 1996. DOI: https://doi.org/10.24432/C5XW20.

[2] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Wine Quality." UCI Machine Learning Repository, 2009. DOI: https://doi.org/10.24432/C56S3T.

[3] T. J. LaGrow, "From SGD to AdamW: History, Math, and Mechanisms." UCI Machine Learning Repository, 2025. `https://gatech.instructure.com/courses/499842/files/folder/Supplemental%20Readings/Week%206?preview=68842381`.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[5] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *arXiv preprint arXiv:1207.0580*, 2012.

[6] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?," *Advances in neural information processing systems*, vol. 32, 2019.

[7] R. M. Zur, Y. Jiang, L. L. Pesce, and K. Drukker, "Noise injection for training artificial neural networks: A comparison with weight decay and early stopping," *Medical physics*, vol. 36, no. 10, pp. 4810–4818, 2009.

Regularization Method	Parameter	Best Val Loss	Test F1	Test Accuracy
L2 weight decay (coupled)	λ=0.001	0.346	0.658	0.837
Early stopping	patience=1000 grad evals, delta=0.001	0.343	0.653	0.839
Dropout	p1=0.3, p2=0.5	0.344	0.657	0.840
Label smoothing	α=0.15	0.360	0.660	0.838
Input noise injection	Gaussian σ=0.15	0.386	0.650	0.837

TABLE III: Regularization studies. Baseline is standard Adam with test F1 = 0.649, test accuracy=0.839

Regularization Combinations	Best Val Loss	Test F1	Test Accuracy
All	0.374	0.641	0.834
- dropout	0.370	0.632	0.837
- label smoothing	**0.352**	**0.622**	**0.836**
- input noise	0.369	0.652	0.838
- coupled L2 weight decay	0.374	0.631	0.835
- early stopping	**0.390**	**0.636**	**0.832**

(a) Part 1: start from all 5 regularization methods with tuned parameters in table III, remove each method and evaluate change in metrics. Find key methods

Regularization Combinations	Best Val Loss	Test F1	Test Accuracy
label smoothing + early stopping	0.361	0.655	0.837
Label smoothing + early stop at 2000	**0.361**	**0.669**	**0.839**
Label smoothing + early stop at 2000 + dropout	0.368	0.667	0.840
Label smoothing + early stop at 2000 + coupled L2 weight decay	0.364	0.663	0.838
Label smoothing + early stop at 2000 + droupout + coupled L2 weight decay	0.366	0.656	0.836
Label smoothing + early stop at 2000 + dropout + coupled L2 weight decay=1e-4	**0.366**	**0.670**	**0.837**

(b) Part2: Iteratively add other regularization methods on top of key methods identified in part 1

TABLE IV: Regularization best combination search